

Steven Ens

TIFDAO Core Review

Security Assessment

June 1st, 2026



Disclosure:

This report format is adapted from publicly available smart contract audit report structures used by **Guardian Audits**, for educational and portfolio purposes. All findings, vulnerability analysis, threat modeling, testing, and remediation recommendations contained in this report are original work. Find more of Guardian Audits work at: <https://github.com/GuardianAudits/Audits>.

Summary:

Audited By: Steven Ens

Client: TIFDAO (tifdao.com)

Final Report Date: June 1st, 2026

Audit Summary:

This is a personal audit performed by Steven Ens on the Token Index Fund DAO's Core smart contracts. All findings have been recorded in the following report.

Confidence Ranking:

Given the

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only Low or Medium-severity issues were discovered. Design patterns are sound, and test coverage is reasonable. Small changes, such as modifying rounding logic, may introduce new vulnerabilities and should be carefully reviewed. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0 High/Critical findings. Varied Low/Medium severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1 High finding and ≥ 3 Medium. Varied Low severity findings.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities (~2/week). Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥5 High/Critical findings and overall systemic flaws.

Disclosure:	2
Summary:	2

H-01 | Stale Signed Data Can Skew Token Rewards

Description:

To reduce the onchain cost of calculating the maximum/minimum valued token at the time of user transaction, a backend server makes a staticcall to retrieve and sign the following parameters:

```
uint256 timestamp,  
address userAddress,  
address minValuedTokenInfo.tokenAddress,  
uint256 minValuedTokenInfo.minTokenOutput,  
uint256 minValuedTokenInfo.tokenPrice,  
uint256 minValuedTokenInfo.tokenAmount,  
uint256 minValuedTokenInfo.tokenValue,  
uint256 runningValueTotal
```

The runningValueTotal is an Ether value of the sum of all token values currently held by the TIFDAO Fund. It is used in `_mintUserContribution()` along with the `totalSupply()` to calculate the percentage of value the user provided to the fund in their transaction before minting the relative amount of tokens.

```
mintInfo.userContributedValuePercent = _divide(  
    mintInfo.userContributedTokenValue,  
    runningValueTotal  
);  
mintInfo.tifToMint = _multiply(  
    mintInfo.userContributedValuePercent,  
    mintInfo.tifTotalSupply  
);
```

If two user transactions are signed and then included within the same block, the second user will receive less tokens than expected as the stale runningValueTotal would be used. However, `mintUserContribution` uses the `totalSupply()` of the token so the first user's would update that value but still use the old runningValueTotal, skewing the rewards they receive and the ratio of tokens to value held by the fund.

Recommendation: Verifying the timestamp last called is

Users A and B both send transactions included in the same block with the following parameters:
(same block matter?)
userContributedTokenValue: 10 Ether
runningValueTotal: 100 ether

Both users therefore have the same:
userContributedValuePercent: 10%

User 1's transaction goes through. The total supply is currently 100 tokens, so tifToMint is equal to 10 tokens.

User 2's transaction goes through. The total supply is currently 110 tokens, but the calculation uses the old runningValueTotal.

The userContributedValuePercent is still 10%, but the minting calculation is now skewed:

$10\% * 110 \text{ tokens} = 11 \text{ tokens}$

Fix:

User2's transaction goes through:

```
// Ensures only one transaction goes through per block otherwise the user must get updated
input data
if (timestamp < _lastTimestampCalled)
    revert OutdatedInputData();
// Update to ensure only one transaction per block
_lastTimestampCalled = block.timestamp;
```